

ATHANASIOS GOURDOMICHALIS ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS Assignment in Computer Programming with C++

Technical report and C++ code analysis

INTERACTIVE APPLICATION USING A NODE NETWORK: REPRESENTATION OF TREE TRAVERSALS IN A USER-CREATED BINARY TREE

This technical report is intended for anyone looking to gain an in-depth knowledge and understanding of the application's underlying C++ code architecture, software implementation details, and the specific [Simple Game Graphics \(SGG\)](#) library mechanisms utilized to achieve the project's core objectives.

- After the introduction, a detailed analysis is provided for each object class (for both the .h C++ header files and the .cpp source code files).

Introduction & Project Scope

The Tree Traversal Simulator is an interactive educational 2D graphics application in C++ designed to facilitate the intuitive understanding of Binary Tree structures and their core traversal algorithms. The software allows users to freely construct binary trees on an interactive canvas, establish parent-child relationships using drag-and-drop mechanics, and follow step-by-step stop-motion animations of Preorder, Inorder, Postorder, and Level-order (BFS) traversals. The implementation is built on top of the [Simple Game Graphics \(SGG\)](#) framework, leveraging its 2D rendering pipeline, event message loop, input polling, and asset management systems for fonts, textures, and audio playback.

1. Application Core & Lifecycle ([main.cpp](#))

- Window & Virtual Canvas Setup:
`graphics::createWindow(1200, 800, "Tree Traversal Simulator")` initializes the native OS window. The virtual drawing canvas is decoupled from native pixel resolution and set to **1500 × 1000** canvas units using `GlobalState::CANVAS_WIDTH` and `CANVAS_HEIGHT`.

- **Aspect Ratio Preservation:**

Configured via

`graphics::setCanvasScaleMode(graphics::CANVAS_SCALE_FIT)`, enforcing uniform scaling with letterboxing/pillarboxing to prevent visual distortion of circular nodes and UI widgets.

- **Event Dispatching Callbacks:**

Registered through `graphics::setDrawFunction(draw)` and `graphics::setUpdateFunction(update)` to bridge SGG's event loop with `GlobalState::getInstance()->draw()` and `GlobalState::getInstance()->update(ms)`.

- **Lifecycle Execution & Teardown:**

`graphics::startMessageLoop()` triggers the active event polling and rendering pipeline. Upon window termination, `GlobalState::getInstance()->cleanup()` safely releases all heap-allocated singleton resources.

2. **Central State Controller: Singleton Pattern ([GlobalState.h](#))**

- **Singleton Architecture:**

Implements a **private** constructor and a **private** static pointer `GlobalState*` instance, providing controlled global access exclusively through the **public** static method `getInstance()`.

- **Scene & Flow Management:**

Tracks the active level via `Levels* currentLevel` and handles level transitions dynamically with `changeLevel(Levels* newLevel)`.

- **Centralized Data Hub:**

Serves as the single point of access for shared application state across levels:

- `buttonPressed(setButtonID() / getButtonID())` for decoupled UI event signaling.

- `traversal (setTraversal() / getTraversal())` to preserve the selected algorithm across levels.
- Fixed canvas dimension constants (`CANVAS_WIDTH = 1500.0f`, `CANVAS_HEIGHT = 1000.0f`).
- Polymorphic Dispatching:
Exposes `init()`, `draw()`, and `update(float dt)` methods that delegate execution directly to `currentLevel`, which in turn polymorphically updates and renders all active `GameObject` entities.

Implementation Details ([GlobalState.cpp](#))

- Dynamic Memory & Destructor (`~GlobalState()`):
Invokes `delete currentLevel`, triggering the polymorphic destructor chain `~Levels()` to ensure zero memory leaks when tearing down the active level.
- Lazy Instantiation (`getInstance()`):
Dynamically allocates the `GlobalState` instance on first request
(`if (!instance)`
 `instance = new GlobalState();`).
- Initialization (`init()`):
Loads the global application typography using `graphics::setFont("assets/font.ttf")` and sets the entry level via `changeLevel(new StartLevel())`.
- Level Switching (`changeLevel()`):
Destroys the currently active `currentLevel` (if one exists), reassigns `currentLevel = newLevel`, and immediately invokes `currentLevel->init()` to instantiate the new level's entities.
- Instance Teardown (`cleanup()`):
Executes `delete instance` and resets `instance = nullptr;` to safely finalize the singleton lifecycle at program termination.

3. Base Class for Levels ([Levels.h](#))

- Base Class for Levels:
Acts as the common base class for all application screens (`StartLevel`, `EditingLevel`, `VisualizationLevel`), holding shared functionality and object management.
- GameObjects Collection:
Uses a `std::vector<GameObject*>` objects to store every visual element and GUI widget displayed on the canvas.
- Polymorphic Calls:
The base `draw()` and `update()` methods, loop through the objects vector and automatically call each object's own `draw()` and `update()` methods without needing to know its exact type.
- Memory Management (`~Levels()`):
The virtual destructor `virtual ~Levels()` loops through the vector and calls `delete` on all heap-allocated objects, preventing memory leaks when changing levels.
- Text Centering Helper:
Provides `getOffset(centerX, fontSize, text)` to calculate the correct starting X coordinate ($x_{\text{start}} = x_{\text{center}} - \frac{\text{length} \cdot \text{fontSize} \cdot 0.45}{2}$), centering text properly because SGG draws text from the bottom-left corner of the selected (x, y) coordinates.

Vector Usage & Implementation ([Levels.cpp](#))

- Adding Objects:
The `addGameObject(GameObject* obj)` method appends new objects to the vector using `push_back()` and immediately calls `obj->init()` to set up their brushes and colors.
- Fast Sequential Traversal:
`std::vector` keeps elements in continuous memory, which makes

looping through all objects very fast during each frame's **update()** and **draw()** loop.

- **Dynamic Sizing:**
Allows adding new objects at the end easily without having to **resize** arrays manually.
- **Drawing Order (Layering):**
Objects are **updated** and **drawn** in the **exact order** they are added, ensuring correct **visual layers** on the canvas (e.g., **Background** → **Grid** → **Nodes/Edges** → **Buttons/Labels**).

4. **Base Interface & Entity Hierarchy** ([GameObject.h](#))

- **Core Interface:**
Serves as the **abstract base class** for any **visual entity** drawn on the **canvas** using its **geometric center coordinates** (*pos_x, pos_y*).
- **Derived Class Ecosystem:**
Inherited by all on-screen elements, including **Background**, **Widget**, **Grid**, **Node<T>**, **Button**, **Label**, and **Box**.
- **State Preservation:**
Employs **const accessor methods** (such as **getX()** and **getY()**) to guarantee that **reading coordinates** does not **accidentally modify** object properties.
- **Pure Virtual Contract:**
Enforces derived classes to **implement their own** **init()**, **draw()**, and **update()** methods, enabling **polymorphic handling** across the entire engine.

5. **Level Selection & Routing** ([StartLevel.h](#) / [StartLevel.cpp](#))

- **Entry Screen:**
Inherits from **Levels** and acts as the **initial menu** where users select

one of the 4 traversal methods (preorder, inorder, postorder, levelorder).

- **Resource Initialization (`init()`):**
Dynamically allocates all scene-specific visual assets and UI buttons using `new` and registers them via `addGameObject()`.
- **Automated Cleanup:**
Does not require a custom destructor, tearing down the level automatically triggers the base `~Levels()` destructor to release all allocated heap memory.
- **Code Reusability:**
Overrides `draw()` and `update()` to invoke the base class implementations (`Levels::draw()`, `Levels::update()`) for batch processing.
- **Level (Screen) Transition:**
Checks the clicked `button ID` from `GlobalState`, once a traversal button is pressed, it stores the selection (`setTraversal()`) and immediately switches to `EditingLevel`.

6. Polymorphic Window Background ([Background.h](#) / [Background.cpp](#))

- **Canvas Centering:**
Instantiated at coordinates (0.0f, 0.0f) to match the base canvas anchor.
- **SGG Integration:**
In its `draw()` method, it invokes `graphics::setWindowBackground(br)` using its configured brush to color the application background for each specific level.
- **Polymorphic Consistency:**
By inheriting from `GameObject`, each level can simply push a `Background` instance into its objects collection, allowing uniform initialization and rendering alongside other game entities.

7. Color Utility Macro ([setColor.h](#))

- **Brush Color Helper:**

A macro helper

`(SETCOLOR(c, r, g, b) { c[0]=r; c[1]=g; c[2]=b; })`

designed to streamline assignments to SGG float[3] color arrays.

- **Code Readability:**

Greatly reduces **repetitive array indexing** syntax across `init()` and `update()` methods, making **brush color configurations** much cleaner.

8. GUI Base Interface ([Widget.h](#))

- **GUI Hierarchy Base:**

An **abstract class** extending `GameObject` that establishes **common properties** for all user interface components (`Button`, `Label`, `Grid`, `Node`).

- **Common UI Properties:**

Encapsulates position (*`pos_x, pos_y`*), default RGB color channels (`r, g, b`), and optional texture path (`image`).

- **Abstract Specification:**

Defines **pure virtual methods** (`draw() = 0`, `init() = 0`, `update() = 0`), acting as a strict architectural blueprint for specialized UI elements.

- **Texture Flexibility:**

Accommodates **plain colored elements** without images (like `Label` where `image = ""`) as well as **textured UI components** (like image-based `Button` instances).

9. Interaction & Spatial Collision Interface ([Interactable.h](#))

- Interface Purpose:

Implemented by interactive screen elements to provide a unified **contains()** method for **mouse collision** and **hit-testing**.

- Point-in-Rectangle Test:

Checks if the mouse cursor (m_x, m_y) lies within rectangular boundaries (used in **Button** and **Grid**):

$$pos_x - \frac{w}{2} \leq m_x \leq pos_x + \frac{w}{2} \quad AND \quad pos_y - \frac{h}{2} \leq m_y \leq pos_y + \frac{h}{2}$$

- Point-in-Circle Test:

Uses **Euclidean distance** to verify if the cursor is hovering inside a circular **Node<T>**:

$$\sqrt{(m_x - pos_x)^2 + (m_y - pos_y)^2} < radius$$

- Disk-to-Disk Overlap Test:

Prevents **visual node superposition** in **Grid** by verifying that **center-to-center distance** is at least $(r_i + r_j)$.

10. Interactive Button Component ([Button.h](#) / [Button.cpp](#))

- Architecture & Multiple Inheritance:

Inherits from both **Widget** (for **graphical UI rendering**) and **Interactable** (for **mouse collision hit-testing**). It encapsulates its dimensions (**width, height**), **center coordinates** (pos_x, pos_y), and a unique **string id** to determine which **user action** was triggered.

- Brush & Asset Initialization (**init()**):

Configures **button_br** with the specified **texture path (image)**, **opacity**, and **fill color** via the **SETCOLOR** macro. It also initializes **shadow_br** with **semi-transparent black fill properties** (**fill_opacity = 0.5f**, **outline_opacity = 0.0f**) to create a **drop shadow** for **non-textured buttons**.

- Hit-Testing (`contains()`):

Implements the `pure virtual contains()` method from `Interactable` by polling the `current mouse state`, converting `window pixels` to `canvas space`, and `verifying if the cursor falls within its rectangular boundary`:

$$pos_x - \frac{width}{2} \leq m_x \leq pos_x + \frac{width}{2}$$

AND

$$pos_y - \frac{height}{2} \leq m_y \leq pos_y + \frac{height}{2}$$

- Hover Scaling Animation (`draw()`):

When hovered (`contains() == true`), the button `magnifies its dimensions by 5%` using `graphics::setScale(1.05f, 1.05f)` to provide `immediate visual feedback`, followed immediately by `graphics::resetPose()` to `isolate the transformation` and `avoid affecting subsequent draw calls`.

- Conditional Drop Shadow (`draw()`):

Evaluates the `presence of an image texture` before rendering:

- Plain Buttons (`image == ""`):

Draws an `offset shadow rectangle` at `(pos_x + 5.0f, pos_y + 5.0f)` using `shadow_br` prior to drawing the `main button body` (used for `UI controls in EditingLevel` and `VisualizationLevel`).

- Textured Buttons (`image != ""`):

Skips the `shadow pass` entirely to preserve the `clean alpha edges` of `custom button textures` (such as the `traversal cards` in `StartLevel`).

- Event Dispatching & Audio Feedback (`update()`):

Dynamically applies `hover color highlights` based on `whether the`

button uses an image or a flat color. When left-clicked (`ms.button_left_pressed`), it triggers sound playback via `graphics::playSound("assets/click.wav", 1.0f, false)` and transmits its identifier (`getID()`) to `GlobalState::setButtonID()`, enabling fully decoupled event handling across levels.

11. Typography & Text Rendering Component ([Label.h](#) / [Label.cpp](#))

- **Architecture & Specialization:**
Inherits from `Widget` as a dedicated GUI text entity, encapsulating on-screen text rendering for user guidance and UI descriptions.
- **Design & Code Cleanliness:**
Prevents arbitrary and hardcoded `graphics::drawText()` calls scattered across level files by providing a clean, reusable component that integrates into the polymorphic `GameObject` pipeline.
- **Brush & Typography Setup (`init()` / `draw()`):**
Configures `label_br` opacity and color channels via `SETCOLOR` during `init()`. In `draw()`, it invokes `graphics::drawText(pos_x, pos_y, size, text, label_br)` using its configured coordinates and font scale.
- **Dynamic Content Update:**
Provides `setText()` and `setX()` to allow real-time label modification during runtime (such as updating the active traversal title in `VisualizationLevel`).

12. Tree Construction Environment & Memory Management ([EditingLevel.h](#) / [EditingLevel.cpp](#))

- **Level Responsibility:**
Inherits from `Levels` and coordinates the graph construction workspace, managing vectors of `Node<char>*` and `Edge*`.

- Entity Initialization (`init()`):
Dynamically allocates all level elements (**background**, action buttons, guide labels, and the interactive **Grid**) and registers them via `addGameObject()`.
- Code Reusability:
Leverages base class implementations (`Levels::draw()`, `Levels::update()`) to automatically render and update all child objects.
- Level Navigation & Actions (`update()`):
Evaluates user interaction via `GlobalState::getButtonID()`:
 - "Back" button: Resets the **button ID** and returns to **StartLevel**.
 - "Clear" button: Re-instantiates **EditingLevel** to completely wipe and reset the canvas.
 - "Visualize" button: Transitions to **VisualizationLevel** by passing the **current node** and **edge vectors**.
- Critical Ownership Transfer (Preventing Dangling Pointers):
The destructor `~EditingLevel()` deletes all objects inside its **nodes** and **edges vectors**. To safely pass the constructed tree to **VisualizationLevel** without having its elements destroyed when `GlobalState::changeLevel()` deletes the old level, `grid->clearVectors()` and `clearVectors()` are called first to detach the pointers before destruction.

13. Generic Graph Node Template ([Node.h](#))

- Generic Design (`template <typename T>`):
Implemented as a template class to support arbitrary node data types (**char**, **int**, **float**) without requiring separate class definitions. Extends **Widget** and implements **Interactable**.

- **Node Properties & Anchors:**

Encapsulates the immutable value `const T id`, a fixed bounding radius (`radius = 40.0f`), state flags (`active`, `hover`), pointers to family relations (`parent`, `leftChild`, `rightChild` node), and dedicated brushes for body, text, and child slots.

- **Point-in-Circle Containment (`contains()`):**

Implements the Euclidean distance check from `Interactable` to detect cursor hover:

$$\sqrt{(m_x - pos_x)^2 + (m_y - pos_y)^2} < \text{radius}$$

- **Child Connection Visualization (`draw()`):**

- Renders the circular body with `graphics::drawDisk()` and the centered data label with `graphics::drawText()`.
- When in edge-connecting mode (`hover == true`), it dynamically renders green 20×20 unit target boxes, below the node for available child slots
(left box at `(pos_x - 30, pos_y + 30)` if `leftChild == nullptr`,
right box at `(pos_x + 30, pos_y + 30)` if `rightChild == nullptr`).

- **Color State Machine (`update()`):**

- **Active(`active == true`):** Switches to a gold/yellow hue when visited during traversal execution.
- **Hover (`contains() == true`):** Darkens the base blue color $(0.7 \times r, 0.7 \times g, 0.7 \times b)$ to indicate mouse focus.
- **Default:** Reverts to the original blue gradient (`gradient_dir_v = 1.0f`).

- **Interactive Dragging:** Exposes `setPosition(new_x, new_y)` to update coordinates in real time during drag-and-drop operations on the grid.

14. Graph Edge Component ([Edge.h](#) / [Edge.cpp](#))

- Immutable Node References: Encapsulates two double-const pointers,
`const Node<char>* const startNode` and
`const Node<char>* const endNode`, ensuring the edge cannot alter the connected nodes' internal state or be redirected after creation.
- Brush Configuration (`init()`): Sets up `edge_br` with an outline width of `6.0f`, `full opacity`, and a dark outline color via `SETCOLOR()` to draw thick connection lines.
- Cubic Bezier Rendering Workaround (`draw()`): Utilizes `graphics::drawBezier()` instead of `graphics::drawLine()` to overcome SGG's inability to configure line thickness/width on standard line primitives.
- Geometric Offsets & Smooth Curvature: Offsets the start and end coordinates along the Y-axis by 40 units
($ep1 = (x_1, y_1 - 40)$, $ep2 = (x_2, y_2 + 40)$) so the line anchors at the outer boundary of each node's circle ($r = 40$).
Intermediate control points
($cp1 = (x_1, y_1 - dist \cdot 0.3f)$, $cp2 = (x_2, y_2 + dist \cdot 0.3f)$)
dynamically compute a gentle vertical curve based on the distance
 $dist = |y_2 - y_1|$.

15. Interactive Tree Construction: Grid Mechanics & Spatial Collision Engine ([Grid.h](#) / [Grid.cpp](#))

- Architecture & Multiple Inheritance:
Extends `Widget` for visual canvas rendering and implements `Interactable` for mouse event handling, serving as the primary interactive workspace in `EditingLevel`.

- **Decoupled Tree Construction:** Encapsulates the entire tree creation logic (spawning nodes, moving elements, and connecting edges), cleanly separating graph building mechanics from the higher-level scene management of `EditingLevel`.
- **Graph Containers & Alphabetical Ordering:**
Maintains `std::vector<Node<char*>>` nodes and `std::vector<Edge*>` edges. Employs an integer order counter to assign consecutive alphabetical IDs (`'A' + order`) to nodes upon creation.
- **Interactive State Tracking & Brushes:**
Configures `grid_br` with the `assets/grid.png` texture and line for active edge dragging. Manages interaction states via specific pointers: `movingNode` (the node being dragged with left-click), `child` (the child node initiating an edge), and `parent` (the target parent node).
- **Memory Lifecycle Responsibility:**
The `~Grid()` destructor does not delete node or edge pointers directly, ownership is maintained by `EditingLevel` (and subsequently transferred to `VisualizationLevel`), avoiding double-deletion bugs.
- **AABB Boundary Checking (`contains()`):**
Implements a Point-in-Rectangle hit-test with an internal 40 unit safety margin to constrain all node placements within the usable grid space:

$$pos_x - \frac{width}{2} + 40 \leq m_x \leq pos_x + \frac{width}{2} - 40$$

AND

$$pos_y - \frac{height}{2} + 40 \leq m_y \leq pos_y + \frac{height}{2} - 40$$

- **Disk Overlap Detection (`intercepts()`):**

Calculates the Euclidean distance between a candidate canvas coordinate (x, y) and every existing node $n_i(x_i, y_i)$ in the **nodes vector**, ignoring the node currently being dragged (`ignore()`):

$$dx = x - x_i, \quad dy = y - y_i, \quad \text{dist} = \sqrt{dx^2 + dy^2}$$

`intercepts() = (dist < n_i->getRadius() + 25.0f)`

This prevents visual superposition and spatial overcrowding by disallowing new node placements too close to existing ones.

- **Rendering Pipeline & Dynamic Bezier Preview (`draw()`):**

Draws the **main workspace rectangle** using `grid_br` (`assets/grid.png`) and executes batch rendering calls for all nodes and edges.

If an **edge creation** is active (`child != nullptr`), it renders a **real-time cubic Bezier preview line** stretching from the child node's center to the current cursor position (m_x, m_y) .

- **Left-Click State Machine (Creation, Dragging & Spatial Repulsion):**

- **Spawn New Node:**

If left-clicked on empty grid space (`!intercepts(mx, my)`), allocates `new Node<char>(mx, my, 'A' + order)`, increments `order++`, and registers it via `addNode(n)`.

- **Select Node to Drag:**

Iterates through **nodes** with `contains()`, if the cursor is over a node, assigns `movingNode = n`.

- **Active Dragging:**

Continuously updates node coordinates to follow the mouse via `movingNode->setPosition(mx, my)`.

- Drop & Repulsion Physics:

On left-button release, if the dropped node overlaps an existing node (`intercepts(mx, my, movingNode) == true`), it applies a repulsion displacement vector of $(+50px, +50px)$ in canvas space to snap it into a distinct position, then resets `movingNode = nullptr`.

- Right-Click State Machine (Parent-Child Edge Connection):

- Select Child Node:

On right-click, finds an unparented node under the cursor (`n->contains() && !n->hasParent()`) and sets `child = n`.

- Candidate Parent & Slot Hover:

While holding right-click, dragging over another node (`n != child`) sets `parent = n` and enables `parent->setHover(true)`, revealing green anchor boxes for available left/right children.

- Target Slot Validation & Edge Binding:

On right-button release over target bounds:

- Left Slot: If the cursor is within $m_x \in [p_x - 45, p_x - 15]$, $m_y \in [p_y + 15, p_y + 45]$ and `parent->getLeftChild() == nullptr`, assigns `parent->setLeftChild(child)`.
 - Right Slot: If cursor is within $m_x \in [p_x + 15, p_x + 45]$, $m_y \in [p_y + 15, p_y + 45]$ and `parent->getRightChild() == nullptr`, assigns `parent->setRightChild(child)`.

- Finalization: If the two nodes (child & parent) are connected, it allocates a new `Edge(child, parent)`, registers it via `addEdge()`, updates `child->setParent(parent)`, disables hover boxes, and clears temporary pointers (`child = nullptr, parent = nullptr`).

16. Visualization Level Architecture & Graph Sanitization ([VisualizationLevel.h](#) / [VisualizationLevel.cpp](#))

- Core Role & Scene State: Serves as the primary stage for animating binary tree traversals, taking ownership of the **nodes** and **edges vectors** constructed in **EditingLevel**.
- Memory Ownership & Destructor (`~VisualizationLevel()`): Directly deallocates all dynamic heap memory for **nodes** and **edges** when the level is destroyed (e.g., navigating back to the home screen).
- Graph Sanitization & Memory Cleanup (`removeLoneNodes()`):
 - Acts as a logical validation pass before visualization to ensure tree structural integrity.
 - Iterates through nodes using an iterator, any floating node with no parent and no children (`!hasParent() && !getLeftChild() && !getRightChild()`) is deleted from heap memory and safely erased from the vector via `iter = nodes.erase(iter)`.
- Dynamic Centering & Initialization (`init()`):
 - Invokes `removeLoneNodes()` to determine the final valid **node count**.
 - Computes the dynamic horizontal starting position for the bottom visualization queue (`startX = (CANVAS_WIDTH / 2.0f) - (nodes.size() * 80.0f / 2.0f) + (80.0f / 2.0f)`), centering the order array on screen.
 - Fetches the chosen algorithm from `GlobalState::getTraversal()`.
 - Configures `box_br` to draw a static grid (a rectangle background image) directly, replacing the interactive `Grid widget` instance which is destroyed with **EditingLevel**.

- Batch Rendering Pass (`draw()`):
Invokes `Levels::draw()` to render background and UI controls, draws the static grid backdrop, and executes fast sequential `draw()` calls for all nodes, edges, and animated boxes.

17. Tree Traversal Algorithms & STL Data Structures ([VisualizationLevel.cpp](#))

- Root Node Identification (`findHead()`):
Searches the `nodes` vector to locate the unique root node (defined as having no parent node and at least one child node:

```
!n->hasParent() && (n->getLeftChild() || n->getRightChild()))).
```

Returns `nullptr` if no valid binary tree structure exists.

• Inorder Traversal (Recursive function - Left → Root → Right):

- Recursively visits the left subtree, appends the current node pointer to `tempNodes`, dynamically allocates a corresponding `Box` entity at $(startX + boxes.size() \cdot 80, 950)$ via `addBox()`, and recursively traverses the right subtree.

• Preorder Traversal (Iterative with `std::stack` - Root → Left → Right):

- Pushes the tree root into a `std::stack<Node<char>*>`.
- In a loop, pops the top element, appends it to `tempNodes`, and registers its `Box` in the UI array.
- LIFO Optimization: Pushes the right child into the stack first and the left child second, ensuring the left subtree is popped and visited before the right subtree (Last In First Out rule).

- Postorder Traversal (Recursive function - Left → Right → Root):

- Executes recursive calls for the left and right subtrees before appending the current node to `tempNodes` and adding its `visualization Box` to the bottom display array.

- Level-order Traversal (Iterative with `std::queue` - Top-to-Bottom, Left → Right / BFS):

- Pushes the tree root into a `std::queue<Node<char>*>`.
- In a loop, dequeues the front element, appends it to `tempNodes`, dynamically allocates a corresponding `Box` entity at $(startX + boxes.size() \cdot 80, 950)$ via `addBox()`, and enqueues the left child followed by the right child to preserve the FIFO level-order rule (First In First Out).
- BFS Queue Architecture:
 - Breadth-First Search (BFS) Traversal: Implements a level-by-level traversal using the standard `std::queue<Node<char>*>` container.
 - FIFO Queue Mechanics: Pushes the root node to the queue. In each iteration, it dequeues the front element, appends it to `tempNodes`, creates its bottom `visualization Box`, and sequentially enqueues the left child followed by the right child. The First-In-First-Out (FIFO) ordering guarantees nodes are processed top-to-bottom and left-to-right across tree levels.

- Visualization Pipeline Initialization (`setUp()`):

- State & Container Reset:
Clears previous execution data by emptying `tempNodes.clear()` and `boxes.clear()`.

- **Algorithm Dispatch:**
Evaluates the active traversal string and triggers the corresponding traversal method (`inorder`, `preorder`, `postorder`, or `levelorder`) using the tree root obtained from `findHead()`.
- **Timer & Step Configuration:**
Resets `step = 0`, enables the animation state (`visualization = true`), and captures the baseline timestamp via `lastTime = graphics::getGlobalTime()`, to prepare for delayed stepping.
- **Scene Update & Dynamic Interaction (`update()`):**
 - **Polymorphic Base Updates:** Invokes `Levels::update()` to process UI elements and also updates active boxes and `tempNodes`.
 - **Scene Navigation & Routing:**
 - **"Back" button:** Resets node active highlights (`setActive(false)`), instantiates `EditingLevel(getNodes(), getEdges())`, detaches local pointers via `clearVectors()`, and switches scene.
 - **"Home" button:** Resets the button state and transitions back to `StartLevel`.
 - **"Start" button:** Checks that `!nodes.empty()` is true and triggers `setUp()` to initiate the step-by-step animation.
 - **Hot-Swapping Traversal Algorithms:**
When any traversal button ("preorder", "inorder", "postorder", "levelorder") is clicked, it updates the global state via `setTraversal()`, it updates the local string, and dynamically recalculates the centered text position of `traversalLabel` using `getOffset()`.

- Non-Blocking Stop-Motion Animation Engine (update()):

- Timer-Based Stepping (1000 ms delay):

Polling `now = graphics::getGlobalTime()` ensures an asynchronous 1-second delay (`now - lastTime > 1000`) per step without blocking the main rendering thread.

- Step Progression Logic:

- Initial Step (step == 0):

Sets `tempNodes[0]->setActive(true)` and `boxes[0]->setActive(true)` to apply the gold highlight, triggers `graphics::playSound("assets/select.wav", 1.0f, false)`, increments `step++`, and updates `lastTime = now`.

- Intermediate Step (step > 0):

Deactivates the previous elements (`tempNodes[step - 1]->setActive(false)`, `boxes[step - 1]->setActive(false)`), activates the current elements at index `step`, plays the `select.wav` sound, increments `step++`, and resets `lastTime = now`.

- Completion (step >= tempNodes.size()):

Ends the animation (`visualization = false`), deactivates the final node/box, and plays the completion audio cue `graphics::playSound("assets/done.wav", 1.0f, false)`.

18. Traversal Queue Display Component ([Box.h](#))

- Architecture & Inheritance: Inherits directly from `GameObject` to represent an individual visual cell inside the bottom traversal sequence queue in `VisualizationLevel`.
- State & Character Data: Encapsulates the character value (`const char letter`), cell dimensions (`width, height`), an active `boolean` state flag, and dedicated SGG brushes (`box_br, text_br`) for rectangular background and text rendering.
- Brush & Gradient Setup (`init()`): Configures `box_br` with a vertical linear gradient (`gradient_dir_v = 1.0f`) using primary and secondary color arrays set via the `SETCOLOR()` macro, alongside setting `text_br` with high opacity for typography rendering.
- Visual State Machine (`setActive()` / `draw()`):
 - Default State (`active == false`): Renders with a standard blue gradient backdrop while awaiting or completing animation steps.
 - Active Highlight (`active == true`): Switches the primary fill color to a vibrant gold/yellow hue, visually synchronizing the queue element with the active `Node<T>` currently being traversed in the tree above.
- Rendering & Centered Glyphs (`draw()`): Draws the cell's outer rectangular body via `graphics::drawRect(pos_x, pos_y, width, height, box_br)` and renders the node's stored letter label at its center using `graphics::drawText()`.